Checklist Estático e Registro de Bugs

Revisão manual do código-fonte e bugs encontrados durante os testes
Sistema de Batalhas PokeSal

Depois de terminar a implementação, o grupo fez uma revisão manual de todo o código em src/main, respondendo o checklist de teste estático abaixo. Em seguida vieram os testes JUnit, que acabaram revelando problemas que a leitura sozinha não tinha pego. Tudo o que foi encontrado, tanto na revisão quanto nos testes, está registrado na segunda parte deste documento, junto com a correção feita.

Checklist de Teste Estático

1. Todas as variáveis estão inicializadas antes do uso? Sim. O próprio compilador Java obriga a inicialização das variáveis locais. Os campos que começam nulos são verificados antes de serem lidos: a condição de status do PokeSal só é exibida quando existe, o item nulo que representa "não escolher mais itens" é tratado na escolha da mochila, e o vencedor da batalha só é lido depois que o laço de turnos termina.

2. Há variáveis declaradas e nunca usadas? Sim. A constante RANDOM da classe Turn ficou sem uso depois que o desempate por SPD passou a ser fixo (BUG-05), e alguns testes tinham imports que não eram usados. Os dois casos foram removidos. A lista de turnos da Battle é preenchida mas não é lida pelo código de produção; decidimos mantê-la como histórico da batalha.

3. Existe código inacessível? Não.

4. Há código duplicado? Sim. Blaze, Overgrow e Torrent são praticamente iguais, mudando só o tipo elemental, e o mesmo acontece com Super Antidote, Burn Heal e Paralyze Heal, que mudam só o status que curam. Os três métodos de buff do PokeSal (ATK, DEF e SPD) também repetem a mesma validação. A lógica em comum foi movida para duas classes base, LowHpAttackBoost para as passivas e StatusHealEffect para os efeitos, e a validação dos buffs passou para um único método privado.

5. Existem erros de sintaxe? Não. O projeto compila sem erros e sem avisos.

6. Há erros de lógica que quebram regras do negócio? Sim. Foram quatro: o empate de SPD sem regra (BUG-01), os buffs que não voltavam ao valor original (BUG-02), o item que podia ser usado mais de uma vez (BUG-03) e as passivas, status e terrenos que nunca eram acionados durante a batalha (BUG-04). Todos estão detalhados no registro abaixo.

7. Há erros de tipagem? Sim. Os atributos de ATK, DEF e SPD são inteiros, e cada buff aplicado convertia o resultado para int na hora, perdendo a parte decimal. Por causa disso, aplicar e depois desfazer um buff não devolvia o valor original (BUG-02).

8. O fluxo de controle é válido (sem loops infinitos, condições impossíveis)? Sim.

9. Outros tipos de erros identificados? Não foram encontrados outros erros.

Registro de Bugs

BUG-01 — Empate de SPD sem regra definida

Onde foi encontrado: análise estática dos requisitos (item 4 do enunciado).

Problema: o enunciado diz que o PokeSal com maior SPD age primeiro, mas não diz o que acontece quando os dois têm o mesmo SPD. Como alguns pares do catálogo têm SPD igual, a ordem do turno ficava sem definição.

Correção: em caso de empate, o PokeSal do primeiro treinador age primeiro. A regra é coberta pelo teste de velocidades iguais em TurnTest.

Status: corrigido.

BUG-02 — Buffs não voltavam ao valor original

Onde foi encontrado: testes JUnit (EffectTest).

Problema: cada buff ou debuff gravava o novo valor do atributo já convertido para inteiro. Quando o valor era ímpar, cortar pela metade e depois dobrar perdia um ponto: um PokeSal com 65 de ATK queimado ia para 32 e, ao ser curado com Burn Heal, voltava para 64. O mesmo acontecia com a paralisia no SPD, com Blaze, Overgrow e Torrent ao desativar e com o buff de velocidade ao expirar, deixando o PokeSal permanentemente mais fraco.

Correção: o PokeSal passou a guardar o valor base de cada atributo, que nunca muda, e um multiplicador acumulado em double. O valor final é calculado só na hora da leitura, com uma única conversão para inteiro. Os testes de Burn Heal e Paralyze Heal em EffectTest e o teste de buff seguido do inverso em PokeSalTest garantem a correção.

Status: corrigido.

BUG-03 — Item podia ser usado mais de uma vez

Onde foi encontrado: revisão do requisito de mochila (item 5 do enunciado) e testes JUnit (TrainerTest).

Problema: o limite de dois itens só era conferido na hora de colocar o item na mochila. A ação de usar o item não verificava se ele ainda estava lá, então o mesmo item podia ser usado de novo depois de já ter saído da mochila, furando o limite por batalha.

Correção: a ação de usar item agora confere se o item está na mochila do treinador antes de aplicar qualquer efeito e lança IllegalStateException quando não está. O TrainerTest cobre o uso repetido, o uso de um item que não está na mochila e a tentativa de uso depois de esvaziar a mochila.

Status: corrigido.

BUG-04 — Passivas, status e terrenos não agiam durante a batalha

Onde foi encontrado: revisão manual do fluxo da batalha.

Problema: as classes de passiva, status e terreno estavam certas, mas o laço da batalha nunca chamava os métodos de entrada em campo, de dano recebido e de fim de turno. Na

prática, Intimidate e Rain Dish não faziam nada, Blaze, Overgrow e Torrent nunca ativavam, Burn e Poison não causavam dano por turno, a Solar Power dava o bônus sem cobrar HP, o Central Flowerbed não curava e o buff de velocidade nunca expirava. Os testes unitários não pegaram isso porque chamavam esses métodos diretamente.

Correção: o caso de uso que conduz a batalha passou a acionar a entrada em campo dos dois PokeSal antes do primeiro turno e, ao final de cada turno, aplica o dano do status, o efeito da passiva, a cura do terreno (só para quem ainda está de pé) e a expiração dos efeitos temporários, verificando o vencedor de novo em seguida. A ação de ataque também passou a avisar a passiva do defensor sobre o dano recebido. Foi criado o RunBattleUseCaseTest, que simula batalhas completas para garantir que tudo isso acontece.

Status: corrigido.

BUG-05 — Variável de sorteio sem uso na classe Turn

Onde foi encontrado: checklist de teste estático (pergunta 2).

Problema: a classe Turn mantinha uma instância de Random que tinha sobrado de uma versão anterior do desempate de SPD e não era mais usada.

Correção: a variável e o import foram removidos.

Status: corrigido.

BUG-06 — Mensagens de erro com concordância errada

Onde foi encontrado: revisão manual do PokeSal.

Problema: as exceções dos buffs de DEF e SPD diziam "o multiplicador nao pode ser negativa".

Correção: as mensagens foram corrigidas para "negativo".

Status: corrigido.

Registro de Bugs da Fase 03

Os bugs abaixo foram encontrados durante a Fase 03, depois da refatoração para testabilidade. A partir desta fase, cada registro indica também o commit em que a correção foi feita.

BUG-07 — Registro de status repetido e ambíguo

Onde foi encontrado: execução manual do jogo depois da refatoração, numa batalha entre dois BulbaSal usando Vine Whip.

Problema: o mesmo evento do registro de batalha, logStatusEffect, era usado em dois momentos diferentes: quando um golpe aplicava o status e quando o status causava dano no fim do turno. Os dois imprimiam a mesma frase, "está sob efeito de Poison", então, num turno em que os dois PokeSal se envenenavam, a frase aparecia quatro vezes sem deixar claro o que tinha acontecido. O registro de fim de turno também era emitido para qualquer status, mesmo os que não fazem nada nesse momento, como o Paralyzed. Com dois PokeSal da mesma espécie, ainda não dava para saber de qual treinador era cada um.

Correção: o registro de batalha ganhou dois eventos: logStatusDamage, para o HP perdido no fim do turno, emitido só quando o status realmente tira HP, e logBattleStart, que permite ao console mostrar o nome do treinador ao lado de cada PokeSal. O logStatusEffect passou a indicar apenas que o status foi aplicado. A correção é coberta pelo ConsoleBattleLoggerTest e por dois testes novos do RunBattleUseCaseTest, que verificam o registro do dano do veneno e a ausência de registro para um status que não tira HP.

Commit: 4abdf94.

Status: corrigido.

BUG-08 — Violações do Checkstyle que passaram despercebidas

Onde foi encontrado: configuração da integração contínua (item 4.5), ao fazer o Checkstyle interromper o build sempre que houver uma violação.

Problema: três violações entraram no código durante a Fase 03 sem serem percebidas: o bloco vazio escrito como "{ }" na ação usada quando o treinador perde a vez, no RunBattleUseCase, e os números 4 e 8 escritos diretamente no menu de tamanho do torneio, no BattleCli. O plugin do Checkstyle estava configurado só para avisar (failOnViolation igual a false), e a verificação era feita com o Maven em modo silencioso (-q), que esconde esses avisos.

Correção: o bloco vazio passou a ser escrito como "{}" e os tamanhos do torneio viraram constantes. O Checkstyle agora interrompe o build quando encontra qualquer violação, e a CI executa essa verificação a cada push e pull request.

Commit: 6cab06c.

Status: corrigido.